CS4243 Lab 2: Perspective-Aware Virtual Advertisement Overlay

Semester 1, 2026-2027
Due: 8 November 2026, 23:59

Objective

Live sports broadcasts replace the physical advertisement boards around a pitch with virtual artwork, frame after frame, while the camera pans and zooms and players run in front of the boards (Figure 1). In this lab, you will build that system for a flat, textured *advertisement board* (a framed poster standing in for a pitch-side board) as a classical computer-vision pipeline that:

1. **locates** the board by detecting, describing, and matching local features (Lectures 7-8);
2. **maps** replacement artwork onto the board with a planar homography estimated by normalised DLT and RANSAC, and warps the artwork into the frame (Lecture 9);
3. **keeps the overlay stable over time** by propagating correspondences with optical flow, re-estimating the homography, and monitoring for failure (Lectures 11-12); and
4. **preserves foreground objects** that pass in front of the board with an occlusion mask and explicit alpha compositing (Lectures 3 and 6).

Develop the pipeline on the common clip. Freeze its settings on validation data, then test the supplied stress clips. Most isolate one capture factor, such as viewpoint, scale or blur; the combined clip contains several factors. Next, test your own recorded clip and ask an open-weight video model to perform the same edit. Compare the results. Keep intermediate evidence for every stage. The supplied tools build three stress tables; Notebook 4 provides the workspace for your personal Overlay Scorecard and comparison.

The Clips

The supplied clips show the same framed poster, filmed hand-held at 1280 x 720 and 30 fps (POT-280 object 27 [7]). They share a rectified poster reference. The manifests retain annotations at 16-frame intervals where valid benchmark corners are available. Excluded frames are not evaluation targets. Figure 2 shows one frame from each clip.

- **Common clip** (Figure 2a): a blue card crosses the poster eleven times under mild hand-held motion; six occlusion masks. This is the clip you develop on: the unit tests use it, and every threshold is chosen on its *validation* half (the even-indexed annotated frames and three of the six masks) and then frozen. Its other half (the odd-indexed frames and the other three masks) is held out as the common clip's *test* rows, which every stress table reports next to the stress clips.
- **Stress clips** (Figure 2b-f): *viewpoint*, *scale*, *blur*, *exit*, and *combined*, test only: each Stage is reported on the common clip and on the two stress clips that stress it, with the configuration frozen. The viewpoint clip has three *empty* masks (nothing occludes the board, so every masked pixel is a false occlusion), the combined clip six card masks. A seventh take (in-plane rotation only) is supplementary.



Figure 1: A pitch-side advertisement board in a live football broadcast (AFF Mitsubishi Electric Cup 2024, Singapore vs. Malaysia). The outlined board is the physical surface; the inset artwork is what a virtual-advertising system renders onto it, frame after frame, with the board's perspective, while the camera and the players move. This lab builds that system for a flat poster.

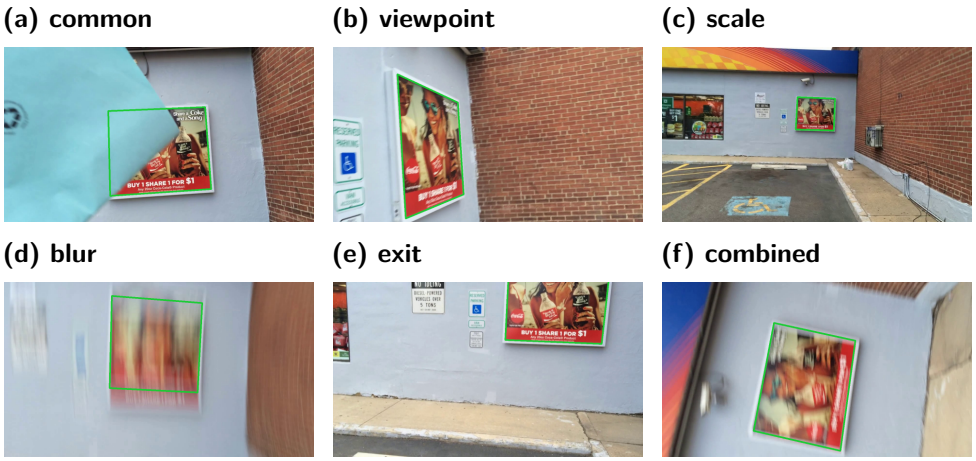


Figure 2: The clips, one frame each, with the ground-truth quadrilateral in green. (a) The blue card crosses the poster. (b) The camera orbits to a side view, so only a full homography fits. (c) The camera retreats until the board is a third of its reference width. (d) Motion blur. (e) The board leaves the frame and re-enters. (f) In-plane rotation up to ±80°, orbit, fast motion, and two card crossings.

- **Artwork**: `ad.png` (1200 × 1000, a staff drawing without third-party brands, in the poster's 6:5 ratio) is used for every run on the supplied clips, so that all tables are comparable; for your own clip, you design your own artwork: a brand-free PNG in your board's aspect ratio with enough distinct texture for the evaluator to re-localise it (requirements under Stage 4).
- **Your clip** (Stage 4): one clip of 5-15 s that you film yourself of a flat, textured board such as a printed poster or a notice board, with a pan, a change of distance, a change of viewing angle, and one occlusion crossing the board. It comes with its own reference image, audit annotations (board corners on ten frames, occlusion masks on five), and artwork; the capture requirements are listed under Stage 4.

Clip identifiers, mask counts, and the data layout are listed in `code/README.md` ("The clips").

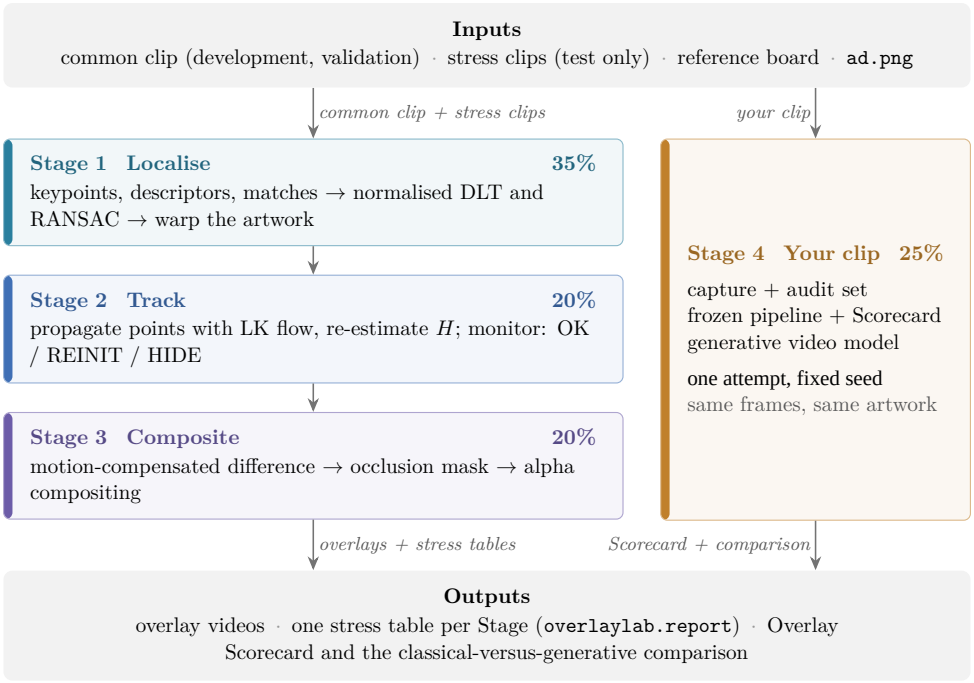


Figure 3: Lab workflow and marks. Each of Stages 1–3 is frozen on the common clip's validation frames and reported on the common clip and its two stress clips; Stage 4 applies the frozen pipeline and a generative video model to your own clip. Audit annotations are recorded before inference and used for evaluation only.

Marks and Workflow

Stages 1-3 form the pipeline of Figure 3, which also shows the marks: Stage 1 Localise 35%, Stage 2 Track 20%, Stage 3 Composite 20%, and Stage 4 Your clip 25%; the subsection headings carry the component marks. Every Stage closes with the same three steps: tune its parameters on the common clip's validation frames and freeze them, run the frozen pipeline on the common clip and the Stage's stress clips, and build the Stage's stress table with `overlaylab.report`. Stage 4 applies the frozen pipeline to your own clip.

What each document covers. This handout says what each Stage is for, how it is marked, and which rules cost marks. The docstring of every `YOUR CODE HERE` function is its specification (steps, return values, status flags); `code/README.md` holds the setup, data layout, clip identifiers, commands, the interfaces table, and the submission tree; `code/REPORT_TEMPLATE.md` lists what every report section and Question must contain.

Stage 1 Localise (35%)

Finding the board in a frame from the reference image alone is the initialisation step of the broadcast system.

Keypoints (10%)

- `harris_response` and `detect_corners` (4%): the Harris response from the supplied Sobel gradients (Lecture 7), with non-maximum suppression in both configured modes: `"window"`, the local maximum of a $(2r+1)\times(2r+1)$ neighbourhood, and `"radius"`, greedy acceptance by response, which spreads the corners when a few high-contrast structures would otherwise dominate.

- `describe_keypoints` (3%): the SIFT descriptor (Lecture 8), computed at a fixed scale on your Harris corners: a 4×4 grid of 8-bin gradient-orientation histograms (128-D) normalised to the keypoint's dominant orientation.
- `match_descriptors` (3%): nearest-neighbour matching with Lowe's ratio test and an optional mutual check.

Return the responses, the kept indices, and the match distances and ratios, so that every corner and match can be visualised and ranked. The parameter sweeps and the `cv2.SIFT` baseline are given as labelled cells in Notebook 1.

Geometry (12%)

Because the board is planar, the reference image and any frame are related by a homography with eight degrees of freedom, and four non-collinear correspondences suffice (Lecture 9).

- `normalise_points` and `dlt_homography` (4%): the normalising similarity and the DLT solved by SVD; refuse fewer than four points, and report a failure status rather than a meaningless matrix when the system is rank deficient.
- `ransac_homography` with `symmetric_transfer_error` (5%): RANSAC around your DLT. Each iteration draws four correspondences with the configured random generator (a sample with three collinear points is rejected), fits a homography to them, and counts as inliers the correspondences whose symmetric transfer error, the squared reprojection error summed over both directions, is below the threshold. The number of iterations adapts to the inlier fraction of the best model so far. Refit the homography on all inliers of the best model, and return a status flag instead of a homography when fewer than `config.min_inliers` survive.
- `transfer_points` and `warp_artwork` (3%): a destination-to-source warp that maps every frame pixel inside the projected quadrilateral back into the artwork and samples it with your Lab 1 `bilinear_sample`, returning the warped image and a Boolean coverage mask (`True` on the frame pixels that received artwork, that is, inside the projected board; it becomes the base of the compositing alpha in Stage 3).

Coordinate convention: a point is (x, y) with x along the columns and y along the rows, and the centre of a pixel sits at integer coordinates, so the corner pixel $(w-1, h-1)$ of the artwork maps to the corner of the board, with no half-pixel offset. Notebook 1 warps a synthetic checkerboard with a known homography and checks that it lands on the expected corners, so that a swapped axis or an off-by-one error shows up before you touch real frames.

Board localisation (8%)

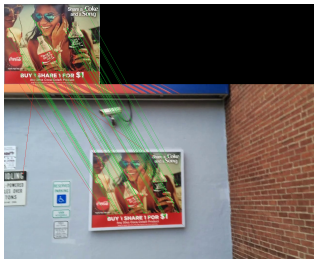
- `localise_board` chains the two branches: keypoints in the reference and the frame, ratio-test matches, RANSAC, and a diagnostics dictionary (counts, inlier ratio, residuals, iterations, status) next to the homography, the projected corners, the matches, and the inlier mask.
- `artwork_to_frame` composes the artwork-rectangle-to-board map with the board-to-frame homography; `overlay_frame` renders one overlaid frame without occlusion handling (Figure 4); `evaluate_audit_frames` summarises the corner errors on annotated frames through `metrics.py`.

Alternative feature settings are selected through the configuration only.

Stress table and Questions 1-2 (5%)

- **Metric.** The *normalised corner error* (a scale-normalised form of the corner alignment error used in planar-tracking benchmarks [7]): the mean distance between the four projected and

(a) Reference-to-frame matches



Green: RANSAC inliers; red: rejected outliers

(b) Inliers and projected corners



(c) Single-frame overlay with our poster



Figure 4: Board localisation on frame 320 of the common clip with the staff reference pipeline (reference stacked above the frame in (a)). RANSAC keeps 49 of 57 ratio-test matches; the homography projects the reference corners with an error of 0.45% of the board diagonal and places `ad.png`. Illustrative, not a reference answer.

annotated corners, divided by the longer annotated diagonal. Choose the success threshold on common-clip validation frames, then freeze it.
Table. python -m overlaylab.report evaluates common (validation/test), viewpoint and scale. It reports median/p90 error, success, inliers, inlier ratio and affine error on the same RANSAC inliers. Both success rates count every eligible frame. Paste the table unchanged.
Evidence. Plot inlier count and ratio against frame index. Compare RANSAC with least squares on all ratio-test matches. Show the two worst cases with matches, residual histograms and quadrilaterals. If fewer than two fail the frozen threshold, say so.

Question 1 (matching). On the scale clip or the viewpoint clip, pick a frame pair on which your matching rate collapses. Which part of the descriptor fails there? Dominant-orientation normalisation absorbs some changes between the two frames. Which ones does it absorb? Which ones does it not absorb? Now look at the scale row of your table. Remember that the reference image has a fixed size. What does this row tell you about a descriptor without scale invariance? Compare with the SIFT cell of Notebook 1.

Question 2 (geometry). On the viewpoint clip, explain from the camera motion why the affine column of your table breaks down as the camera orbits the poster while the homography does not. Warp a whole frame with the board's homography: the board aligns and the background does not, because a homography is exact only for points on the board's plane; a point in front of or behind that plane lands elsewhere by an amount that grows with its distance from the plane and with the camera's translation. This residual shift is parallax. Why does it not harm the overlay, and under which capture condition would it?

Stage 2 Track (20%)

Running Stage 1 on every frame is the simplest temporal strategy: the whole keypoint pipeline runs on every frame, and nothing links one estimate to the next, so each carries its own noise and the overlay may jitter. Broadcast systems instead track features between frames with optical flow and re-estimate the homography from the tracked points (Lecture 11). Whether tracking is faster and smoother on the common clip, and what it costs in drift, is what your Stage 2 table measures. The code for this stage is in `temporal.py`.

Lucas-Kanade flow (6%)

`lk_flow` is an iterative single-level Lucas-Kanade estimator at sparse points: a $(2w+1)\times(2w+1)$ window, the supplied Sobel gradients, and bilinear sampling of the warped window at every iteration. Return the displacements, the two eigenvalues of the window's second-moment matrix, the final residual, and a validity flag that is false when the smaller eigenvalue is too small (the aperture problem on a straight edge, or a textureless window), when the residual is too large, when the iteration does not converge, or when the point leaves the image. A pyramid built with `supplied.gaussian_pyramid` is an optional extension; `cv2.calcOpticalFlowPyrLK` appears only in the labelled cell of Notebook 2. Single-level LK is sufficient for this lab: with everything else frozen, a pyramidal LK or a learned dense flow such as RAFT tracks more frames of the exit clip, but none rescues the blur clip or lowers the corner error, because the limit is re-localisation on blurred frames and the monitor policy, not the flow (Question 3). You may add such a comparison to the same labelled cell; it does not replace `lk_flow`.

Two temporal variants (5%)

`track_board` runs either variant over a clip, selected by the configuration: *independent registration* (`localise_board` on every frame from the reference image) or *flow-assisted registration* (propagate the inlier correspondences with `lk_flow`, drop invalid points, re-estimate the homography with RANSAC, and reinitialise from the reference under a documented rule: too few inliers, a fixed interval, or the monitor firing). Both return a per-frame homography, corners, status, and the monitor statistics; the flow-assisted variant also exposes the motion vectors, eigenvalues, and validity decisions of every frame. A KLT-style tracker that keeps a target state through temporary loss (Lecture 12) is an optional extension.

Failure monitoring (4%)

A deployed overlay must know when to hide itself. `failure_monitor` turns the per-frame statistics (inlier count and ratio, reprojection error, corner displacement, mean flow, brightness change) into one flag per frame, OK, REINIT, or HIDE, with the statistic that triggered it; the docstring fixes the rules and their precedence. Thresholds are chosen on the common clip's validation frames and frozen. Compare projected corners and reprojection errors, never homography matrices, which are defined only up to scale. Figure 5 shows the monitor at work.

Stress table and Question 3 (5%)

- **Table.** The common clip (validation and test, both variants), the blur clip, and the exit clip (flow-assisted), built by `overlaylab.report` from the statistics files of your runs: corner error and success, *jitter* (median frame-to-frame corner displacement after subtracting the interpolated ground-truth motion), *drift* (median corner error at least 16 frames after the last reinitialisation, next to the longest run carried without one), frames tracked, and the REINIT and HIDE counts.
- **Evidence.** One sequence in which optical flow stabilises the overlay and one in which it fails through occlusion, aperture ambiguity, illumination change, blur, or accumulated error; the blur clip or the combined clip supplies the second.

Question 3 (tracking). Does independent registration jitter more than flow-assisted registration on the common clip, and does flow-assisted registration drift? Explain both answers from your table, including the number of reinitialisations. On the blur clip, use the two eigenvalues at

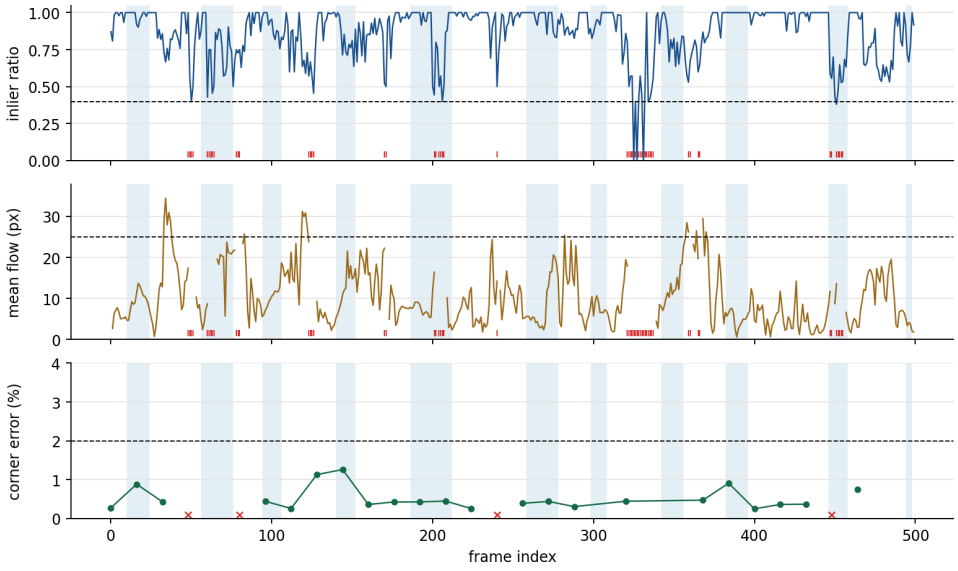


Figure 5: Failure diagnostics of the staff reference pipeline (flow-assisted) on the common clip. *Top:* inlier ratio over the valid propagated points (threshold 0.40). *Middle:* mean flow magnitude (threshold 25 px; the gaps are reinitialisation frames). *Bottom:* corner error on the annotated frames, an audit measure (crosses: overlay hidden). The ratio dips at every crossing and the overlay is hidden on 53 frames; the flow spikes come from the hand-held motion, not from the card. Illustrative, not a reference answer.

your tracked points to separate aperture-problem failures along the board's straight edges from brightness-constancy failures, and show where each lies on the board. On the exit clip, read the HIDE and REINIT sequence off your table and the monitor timeline. On the combined clip (the run you make for Stage 3), which statistic fires first at the card crossing and at the fast rotation, and how does the monitor tell an occlusion from drift when both lower the inlier ratio?

Stage 3 Composite (20%)

Occlusion mask and compositing (15%)

The artwork must appear behind objects crossing the board. Warp the reference into the frame and compare it with the observed board. Large compensated differences are candidate foreground pixels. Misalignment, blur, reflections and lighting can also cause differences. Implement this motion-compensated differencing in compositing.py:

occlusion_mask runs these steps:
1. Warp the reference into the frame.
2. Call compensated_difference. Fit gain and offset on agreeing pixels, then compute mean absolute colour difference inside coverage. This reduces differences caused by approximately uniform lighting changes; it does not remove every appearance change.
3. Apply two thresholds and your Lab 1 hysteresis (Lecture 3).
4. Apply morphological opening and connected components. Drop components smaller than config.min_area. The docstring specifies the opening.
5. Return the mask, difference map and component statistics.

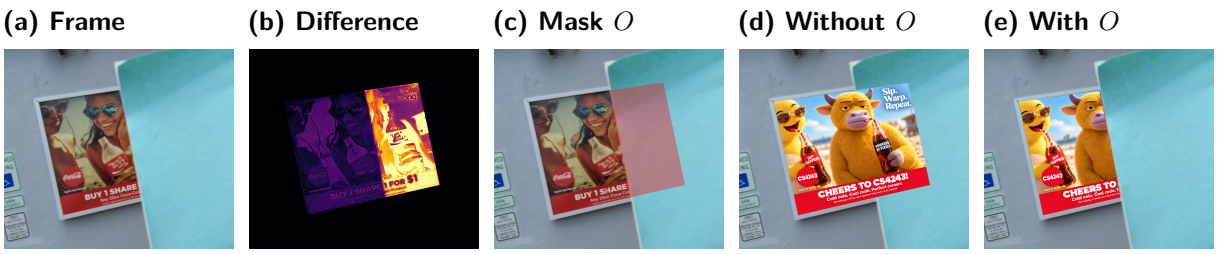


Figure 6: Occlusion handling on frame 144 of the common clip (crops; staff reference pipeline), where the card covers 39% of the poster: (a) the frame, (b) the compensated difference (gain 1.20, offset −0.17), (c) the mask O after hysteresis and cleaning (IoU 0.94 against the annotated mask), (d) the composite without occlusion handling, artwork painted over the card (leakage 99%), and (e) with it, the card in front (0.9%). Illustrative, not a reference answer.

- `select_difference_thresholds` chooses the two levels and the minimum area, on the common clip's validation masks only.
- `composite` blends the warped artwork with the frame through an explicit per-pixel alpha, coverage times one minus occlusion times an optional feathering term, and returns the frame and the alpha map (Figure 6).

A colour-histogram mode is an optional ablation; a promptable segmenter (Lecture 6) may only initialise the mask on the first occluded frame, tagged [AI-VISION], never on the audit frames.

Stress table and Question 4 (5%)

Table. Report common validation/test masks, combined and viewpoint. IoU [16] and F1 [17] compare saved masks. Finished-video leakage estimates foreground pixels painted; compare the common no-occlusion baseline. On empty viewpoint masks, false occlusion is 1 minus visible-board coverage, a pixel-change proxy. It also includes feathering, resampling and similar colours. Exclude absent overlays from coverage; report scored, hidden and missing counts.
Evidence. Explain one error using the difference map, components or flow. Show boundary behaviour on a common and a combined crossing (Notebook 3).

Question 4 (occlusion). Which occluders does motion-compensated differencing recover and which remain ambiguous: an occluder coloured like the board, a shadow on the board, a specular highlight, an occluder that stops moving? How do the mask and the leakage change when the strong threshold or the minimum component area is raised, why does the hysteresis step matter for thin occluders such as a pole, and why does the opening radius you froze work against it? Why does the viewpoint clip produce false occlusion under strong perspective even with a correct homography, and which part of the differencing step is responsible?

Stage 4 Your Clip (25%)

By now you have frozen your pipeline and run it on supplied clips. This Stage runs the same frozen pipeline (unchanged), on your own footage it has never seen, and then asks whether a generative model can do the same job on the same video.

Capture and audit annotation (5%)

Film one clip of a flat, textured board, such as a poster fixed to a wall, a framed poster or a notice board. Cropping is permitted. Meet these requirements:

- Duration: 5-15 s. Use landscape and constant frame rate; recommended 1280 x 720 at 24 fps or higher. Fix exposure/focus where possible.
- The board covers at least 15% of the first video frame and stays visible for most of the clip.
- Include a pan, changes of distance and viewing angle, and an occlusion crossing the board (for example a ball, bag or arm; no faces).
- Exclude identifiable people, private documents, licence plates and precise locations. Obtain consent for any arm or body shown. Remove metadata.
- Save a rectified board as data/personal/<clip_id>/reference.png, at roughly its on-screen width. The descriptor is not scale invariant.
- Save your own artwork as data/artwork/<name>.png in the board aspect ratio. Use no third-party brands, logos or faces. Spread distinct high-contrast structures across it so the evaluator can re-localise it. Include identifiable text or a motif.

Before running any method, annotate corners on ten distributed frames and masks on five of those ten frames. At least three masks must show an object blocking the board. Record capture conditions, difficult events and who captured/annotated the clip. Use the supplied templates and annotation tool. Include the clean, occluded and strongest-perspective comparison frames among the ten corner frames; the occluded comparison frame also needs a mask. Select the temporal pair before generation.

Classical run and Overlay Scorecard (5%)

Run both temporal variants on your clip without adapting the frozen settings. Score both videos (--clip <clip_id>) and fill one Overlay Scorecard using Figure 7. Include a representative overlay and a failure frame with intermediate evidence. If none fails the frozen threshold, show the most challenging frame and say so. Part I of Notebook 4 is an open workspace; use visualisations that support your claims.

Controlled generative comparison (15%)

Use an open-weight video model to generate the edited pixels directly. Inference scripts may prepare inputs, run the model and save output. Do not use generated code or a classical overlay to perform this edit. Compare it with your classical output on the same frames. Examples include MiniMax-H3 [10], JoyAI-Video-Edit [11], LTX-2.3 [12] and Wan2.1-VACE-14B [8, 9]; other open-weight models are welcome. Choose a model your hardware supports. Wan2.1-VACE-1.3B [8, 9] is a smaller option; memory use depends on settings. The grade rests on the comparison.

Inputs. Supply your clip, artwork and frozen-pipeline board masks if required. Notebook 4 exports the masks. Choose three distinct audit frames (clean, occluded, strongest perspective) and one consecutive pair before generation.
Duration. Use a cut containing these frames, or consecutive segments with the same model, settings and seed. Log and preserve every segment; join in source order. Keep the temporal pair within one segment and choose it to map to distinct output frames at the model frame rate.
Timing. Extract at nominal source timestamps, adjusted for any cut, and keep the source indices. Sampling does not undo retiming. Report mismatches; do not search for more favourable output frames.

Overlay Scorecard

Sample output — the staff reference pipeline on the common clip, frozen thresholds, artwork `ad.png`

Clip	common clip, 16.7 s, 1280 × 720 @ 30 fps; framed 6:5 poster covering 8-18% of the frame; hand-held pan with distance and angle change; the blue card crosses eleven times (first at frames 10-24, last at 494-498)
Initialisation	750 keypoints on the reference, 793 on frame 0; 124 ratio-test matches; 108 inliers (0.87); median residual 0.78 px
Geometry	median corner error 0.36% (independent), 0.41% (flow-assisted) on the 13 test frames; success 12/13 for both; the failed frame is card-covered
Temporal	jitter 9.4 px for both variants (the benchmark annotates every 16th frame, so hand-held shake between annotations inflates it); drift 0.88% at least 16 frames after a reinitialisation, longest flow-carried run 22 frames; 111 reinitialisations
Occlusion	IoU 0.85, F1 0.91, leakage 0.6% on the three test masks (frame 448 hidden, not scored); baseline leakage 61%
Monitor	HIDE on 53 frames in twelve episodes (48-51, 60-64, . . . , 321-337 the longest); REINIT triggered 86 times by brightness change, 15 by mean flow, 10 by corner displacement
Description	The artwork stays anchored through the pan and the card is masked on every scored mask frame; on frame 144 the card's cyan edge is close to the poster's sky and a thin strip along it leaks.
Uncertainty	Thresholds were frozen on 13 validation frames; the jitter figure is dominated by the 16-frame annotation stride rather than by the overlay.

Figure 7: Example Scorecard from the staff common-clip pipeline. Fill these rows for your own clip and include source/overlay frames, the mask and monitor timeline. Of three eligible test-mask frames, IoU/F1 exclude hidden frame 448; video leakage includes all three. Report the relevant denominator for each metric.

Prompt. The model receives your clip (or your cut of it), your artwork, and a short instruction such as "Replace the advertisement on the board with the attached artwork.", plus the board masks if it needs a mask. This asks whether the model can carry out the intended edit without explicit geometric or preservation instructions.

Protocol.

- Submit one attempt. Keep settings/seed fixed across segments; preserve all raw outputs.
- Log every invocation in manifests/genai_log.csv: model, version, seed, settings, prompt, interval, time and compute. Log technical retries unchanged.
- Save raw MP4s and frame_XXXXXX.png audit extracts in outputs/genai/. Use [AI-GENERATION]. See genai_conditions.json for the prompt.

Evaluation. Use --frames for both methods on identical selected indices and the same complete audit dictionary. The evaluator measures placement using SIFT [2] and RANSAC [4], rectified artwork fidelity, preservation outside the board (PSNR [13], SSIM [14], background drift), and occlusion leakage and visible-board coverage. It measures flicker, warping error [15] and jitter on the selected pair. Report the number of scored, hidden and missing mask frames with the means. Low leakage alone is insufficient: a hidden overlay paints nothing.
Compare measured results side by side. Background PSNR near a re-encoded source baseline indicates little background change; PSNR alone does not prove how pixels were produced. Treat one generation as a qualitative probe, not a general ranking. The staff reference in data/genai_oracle/ contains the common-clip MiniMax-H3 output [10] and its evaluator report.

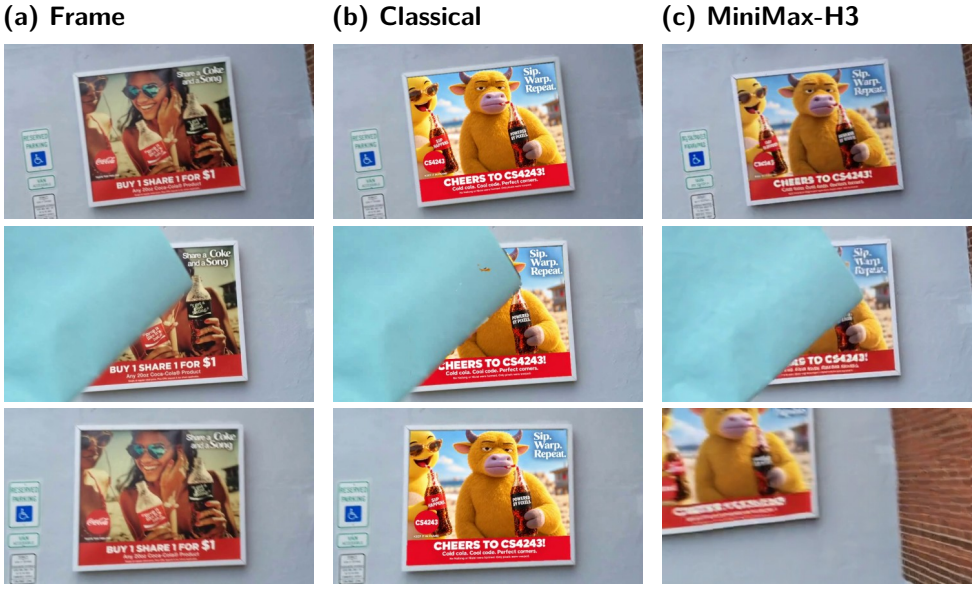


Figure 8: Reference for the generative comparison (`data/genai_oracle/`): MiniMax-H3 [10] on a 5.9 s cut of the common clip (frames 160–335), one run, audit frames 160, 192, and 320 from top to bottom (crops around the poster), next to the classical pipeline. The model reproduces the artwork and keeps the card in front on frame 192 (leakage 8%, classical 0.4%), with median placement error 2.7% (classical 0.4%). It re-renders the whole frame (background PSNR 20 dB, classical 39 dB) and re-times the camera motion, so its frame 320 no longer matches the source. The run took about 36 minutes on one 24 GB GPU.

Figure 8 illustrates the staff example: the artwork is reproduced, but the model changes background pixels and retimes the camera motion.

Question 5 (reflection). Compare the classical pipeline and your generative model on your clip along geometric placement, artwork identity, occlusion order, and background preservation; then discuss controllability when the artwork or its text changes and cost in effort, compute, and time. Name one failure shared by both methods and one specific to each, and state what the preservation and occlusion numbers reveal about what the model actually did, in particular what it generated outside the board and in front of it.

Logistics, Data, and Transparent AI Usage

Working with the Code

Complete every YOUR CODE HERE region in the five assessed files and required cells of Notebooks 2-3. Preserve public signatures. Table 1 lists permitted utilities. Two labelled demonstrations are also permitted: cv2.warpPerspective in Notebook 2 section 6, and cv2.getPerspectiveTransform for evaluation-only GT-fallback in Notebook 3 section 6. Name other utility routines in the report.

Run Notebooks 1-4 from top to bottom after restarting the kernel; Notebooks 1-3 use a resized subset of the common clip so that intermediate results can be inspected quickly. `overlaylab/synthetic.py` renders a board under known homographies with injected outliers, illumination gradients, blur, and moving occluders: debug each Stage there against known answers before touching real footage.

Stage	You must implement	Supplied or permitted
Keypoints	`harris_response`, `detect_corners`, `describe_keypoints`, `match_descriptors`	`supplied.py`; `cv2.SIFT` in the labelled cell of Notebook 1 only
Geometry	`normalise_points`, `dlt_homography`, `ransac_homography`, `symmetric_transfer_error`, `transfer_points`, `warp_artwork`	`numpy.linalg`; `cv2.findHomography`, `cv2.warpPerspective` in the labelled cells of Notebook 1 only
Tracking	`lk_flow`, `track_board`, `failure_monitor`	`cv2.calcOpticalFlowPyrLK`, `cv2.goodFeaturesToTrack`, a learned flow such as RAFT, in the labelled cell of Notebook 2 only
Compositing	`compensated_difference`, `hysteresis`, `select_difference_thresholds`, `occlusion_mask`, `composite`	`cv2.morphologyEx`, `cv2.connectedComponentsWithStats`, your Lab 1 `hysteresis`
Metrics	Nothing; call `overlaylab/metrics.py`	`metrics.py`, `evaluate.py`, `report.py`, supplied and tested

Table 1: What you must implement and what is supplied or permitted, per pipeline component.

Running the Pipeline and Building the Stress Tables

After Notebook 3 has frozen outputs/thresholds.json, run these two commands from code/ to generate the full-resolution videos and stress tables:

```
python -m overlaylab.run_pipeline --all --thresholds outputs/thresholds.json
python -m overlaylab.report
```

The first produces the eight supplied-clip videos with the frozen thresholds and `ad.png` (the common clip with both temporal variants and the no-occlusion baseline; each stress clip flow-assisted; about 30 minutes), each with its statistics file and the masks of its ground-truth mask frames. The second writes `outputs/stress_tables.md`, one table per Stage with the same rows everywhere (common clip validation, common clip test, the Stage's two stress clips). Paste the tables unchanged; the grader rebuilds them from your runs. Videos and still frames are scored with

```
python -m overlaylab.evaluate --video <output.mp4> --clip <clip_id> --artwork <name>
python -m overlaylab.evaluate --frames <frame_XXXXXX.png ...> --clip <clip_id> --
    artwork <name>
```

from the output, the clip, the ground truth, and the artwork alone, so classical, generative, and staff outputs are measured identically; keep the JSON reports in `outputs/evaluation/`.

Published Data Boundaries

Choose settings and prompts on common-clip validation or synthetic data only. Freeze each Stage before test/stress reporting; later Stages must not retune earlier ones from test evidence. Ground truth is for tuning on validation and evaluation, never for pipeline initialisation or correction on test frames. Notebook 3 section 6 has one labelled evaluation-only GT-fallback; it must not enter submitted videos or stress-table predictions.

Manifest	Validation (tuning)	Test (reporting)
`common_clip_corners.csv` (26 annotated frames)	13 even-indexed frames	13 odd-indexed frames
`common_clip_masks.csv` (6 masks)	3	3
`diagnostic_corners.csv`, `diagnostic_masks.csv` (stress clips)	none	all
`audit_corners.csv`, `audit_masks.csv` (your clip)	none	10 corner frames, 5 masks

Transparent AI Usage

AI tools may be used, but every material contribution must remain attributable and independently checked. Place the applicable marker at the end of the paragraph, caption, notebook explanation, or code comment it concerns; group repeated routine uses into one disclosure-table row that identifies the affected files, cells, or figures. The marker set extends Lab 1 with [AI-GENERATION].

Marker	Meaning
[AI-CODE]	AI generated, debugged, translated, or substantially refactored code.
[AI-DESIGN]	AI proposed hypotheses, experiments, prompts, or metrics.
[AI-VISION]	A vision model produced masks, labels, scores, embeddings, or captions.
[AI-GENERATION]	A generative model created or edited image or video pixels.
[AI-LANGUAGE]	An LLM transformed structured evidence into language.
[HUMAN-DATA]	A student captured, selected, or annotated data.
[HUMAN-CHECK]	A student verified, rejected, corrected, or stress-tested an AI contribution using independent evidence.

Tags describe the source, not correctness: pair an AI-produced result with [HUMAN-CHECK] only after checking it against tests, annotations, metrics, or frames, never merely because it was read or watched. Personally captured or annotated clips use [HUMAN-DATA]; a frame returned by the generative model is [AI-GENERATION], never [HUMAN-CHECK] unless you measured it against the audit frames. Complete the AI-use disclosure table in `code/REPORT_TEMPLATE.md` and keep all prompts, raw generative outputs, and retries in the appendix; "AI was used" without locations and verification evidence is insufficient. AI-assisted code remains your responsibility.

Testing and Reproducibility

Before submission, run:

```
python -m unittest discover -s tests -v
```

Passing tests are necessary but not sufficient: notebooks, videos, tables, and written analysis carry the assessment. Implementations must operate on unseen frames without hard-coded paths, file names, dimensions, corner positions, or expected outputs; record configuration values and seeds, including the RANSAC generator. Grading runs the evaluator on every submitted video and frame, the interface and forbidden-call checks, the unit tests, a rebuild of the stress tables, and a short re-run of your pipeline that must reproduce your submitted statistics.

Submission Checklist

Submit one zip with two folders, laid out as in the release: `code/`, with the structure given in `code/README.md` ("Submission"), and `data/`, holding only the files you created, `data/personal/<clip_id>/` and `data/artwork/<name>.png`, not the supplied data. Add any other files only in `code/student_files/`. Before submitting, verify that:

- every `YOUR CODE HERE` region, the required cells of Notebooks 2-3, and the analysis code of Notebook 4 are complete; no interface or file name has changed; the tests and Notebooks 1-4 run from a fresh process; `README.md` reproduces the videos;
- the eight supplied-clip videos use the frozen thresholds and `ad.png`, and `outputs/stress_tables.md` was generated from them, not edited;
- ground truth was used only for validation-frame selection and evaluation, and every comparison changes only the stated factor;
- your clip meets the capture and privacy rules, its metadata is removed, and every audit annotation predates any pipeline output;
- the Scorecard, the generative output, its log, and the evaluator reports are present and traceable; and
- every figure and number names its clip, frame subset, and variant; the questions are answered; the AI markers and the disclosure table are complete.

The report is a major part of your evaluation as it reflects your understanding.

References

[1] C. Harris and M. Stephens, "A Combined Corner and Edge Detector," in *Proceedings of the Alvey Vision Conference*, pp. 147-151, 1988.

[2] D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91-110, 2004.

[3] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2003 (Chapter 4: Estimation, 2D projective transformations).

[4] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381-395, 1981.

[5] B. D. Lucas and T. Kanade, "An Iterative Image Registration Technique with an Application to Stereo Vision," in *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 674-679, 1981.

[6] J. Shi and C. Tomasi, "Good Features to Track," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 593-600, 1994.

[7] P. Liang, H. Ji, Y. Wu, Y. Chai, L. Wang, C. Liao, and H. Ling, "Planar Object Tracking Benchmark in the Wild," *Neurocomputing*, vol. 454, pp. 254-267, 2021 (preliminary version in *ICRA* 2018). Data: https://liangp.github.io/data/pot280/

[8] Team Wan, A. Wang, B. Ai, B. Wen, *et al.*, "Wan: Open and Advanced Large-Scale Video Generative Models," arXiv:2503.20314, 2025.

[9] Z. Jiang, Z. Han, C. Mao, J. Zhang, Y. Pan, and Y. Liu, "VACE: All-in-One Video Creation and Editing," arXiv:2503.07598, 2025. Model: https://huggingface.co/Wan-AI/Wan2.1-VACE-1.3B-diffusers

[10] MiniMax, "MiniMax-H3," open-weight model release and model card, MiniMax H3 Community License, 2026. https://huggingface.co/MiniMaxAI/MiniMax-H3

[11] JD Open Source, "JoyAI-Video-Edit: Real-Time Open-Ended Video Editing with Autoregressive Diffusion," code, technical report and checkpoints, Apache 2.0, 2026. https://github.com/jd-opensource/JoyAI-Video-Edit

[12] Lightricks, "LTX-2.3," open-weight model and in/outpainting IC-LoRA, LTX-2 Community License, 2026. https://docs.ltx.io/open-source-model/feature-guides/editing-effects/in-outpainting

[13] Q. Huynh-Thu and M. Ghanbari, "Scope of Validity of PSNR in Image/Video Quality Assessment," *Electronics Letters*, vol. 44, no. 13, pp. 800-801, 2008.

[14] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600-612, 2004.

[15] W.-S. Lai, J.-B. Huang, O. Wang, E. Shechtman, E. Yumer, and M.-H. Yang, "Learning Blind Video Temporal Consistency," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 170-185, 2018.

[16] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL Visual Object Classes (VOC) Challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303-338, 2010.

[17] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed., Butterworths, 1979.